



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Maven Basics

BUILD LIFE CYCLE, PLUGINS, RUN MAVEN BUILDS, COMPILING AND EXECUTING



Contents

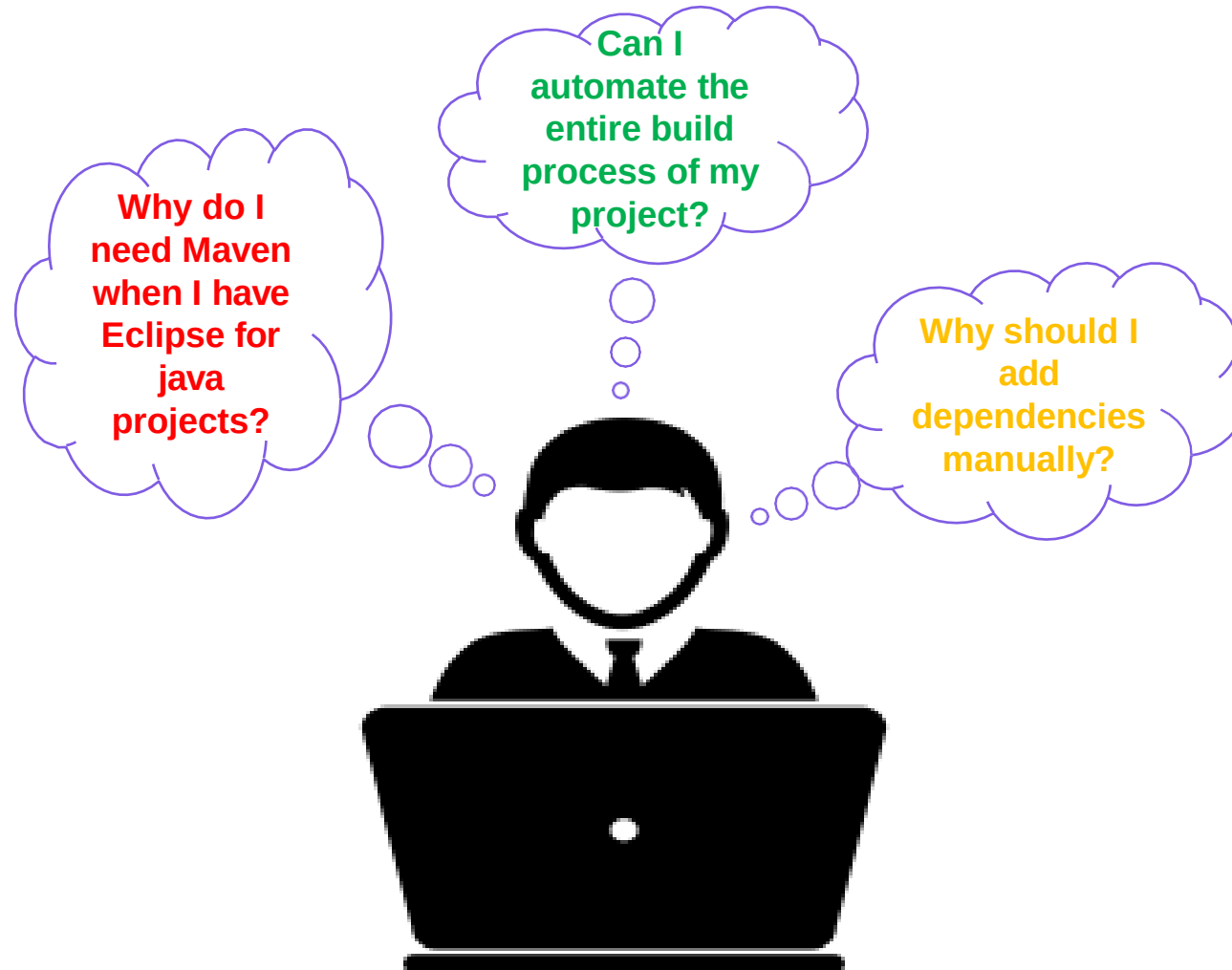
- What is Maven and Why Maven ?
- Key Features
- Maven life cycle Plugin, Goals and Repositories
- Maven architecture
- How to download and install Maven
- Create and build a project from the command line
- Create and build maven projects from eclipse
- Understand the Project Object Model

What is Maven?

- Maven is an open-source **build automation tool and project management framework** primarily used for **Java-based projects**.
- It was developed by the **Apache Software Foundation** .
- It **simplifies** the entire **build process by automating tasks** such as:
 - Compiling source code
 - Running tests
 - Packaging code into distributable formats like JAR or WAR
 - Managing project dependencies



Why do we need Maven?



Key Features of Maven

1. Project Object Model (POM):

- Centralized **XML file (pom.xml)** that contains **project configuration, dependencies, plugins, and build instructions.**

2. Dependency Management:

- Automatically **downloads and manages external libraries and dependencies from remote repositories** like **Maven Central.**

3. Build Lifecycle:

- Provides a standard **set of build phases (e.g., compile, test, package, deploy).**

4. Plugin System:

- Extensible through a **wide range of plugins** for tasks like code compilation, testing, packaging, and deployment.

5. Multi-Module Support:

- Simplifies management of large projects with **multiple submodules.**

Build Life Cycle

- **Core concept** that outlines the **sequence of phases and goals** required to **build and manage** a Maven project
- The process for **building and distributing a particular artifact (project)** is clearly defined.
- Maven provides three built-in build life cycles
 - **Clean Lifecycle:** Focuses on **cleaning the project (removing previous build files)**.
 - **Default (Build) Lifecycle:** Handles the **complete build process, from validation to deployment**.
 - **Site Lifecycle:** Manages the **creation and deployment of project documentation**.

Build Phases

- The **default-lifecycle** executes the following **build phases** in sequential order:
 1. **Validate** : Ensures that the project is correct and all necessary information is available
 2. **Compile** : Compiles the source code
 3. **Test** : Runs **unit tests** using a framework like JUnit
 4. **Package** – Packs the compiled code into a distributable format, such as a **JAR,WAR**
 5. **Verify** – Runs checks to ensure quality criteria are met
 6. **Install** – Installs the packaged application in the local Maven repository.
 7. **Deploy** – Deploys the application to a remote repository.

Maven Build Lifecycle in Real Projects

- Maven has three main lifecycles:
 - 1 Clean Lifecycle – cleaning old builds
 - 2 Default Lifecycle – compiling, testing, packaging, deploying
 - 3 Site Lifecycle – documentation and reporting

We focus mainly on the Default Lifecycle, which drives CI/CD automation.

Example: When you run any Maven command like:

mvn test

- Maven executes all previous phases in sequence: **validate** → **compile** → **test (including clean if**

combined with mvn clean test)

validate phase

What it does:

- Checks if the project structure is correct.
- Validates that all required files (pom.xml, src/main/java, src/test/java) exist.
- Ensures the dependencies are correctly defined and downloadable.
- Prepares the build environment.

In a real IT project: When a developer commits new code to GitHub and Jenkins triggers a build:

- Jenkins executes mvn validate first.
- Maven checks if the pom.xml syntax is valid and that dependencies (Selenium, TestNG, Log4j, etc.) exist in the local or remote repository.
- If a developer forgets to update the version in pom.xml, this phase will catch it before compiling.

Maven Basics

validate phase

Example: In a Selenium project, it ensures:

- TestNG dependency is available.
- Selenium version is compatible.
- Folder structure follows convention:

```
src/main/java/  
src/test/java/
```

Maven Basics

clean phase

What it does:

- Deletes previously compiled .class files and old reports in the /target folder.
- Ensures a fresh environment before a new build.

In a real IT project: Before each Jenkins job or manual build, teams run:

mvn clean

This ensures:

- No leftover screenshots, old logs, or compiled .class files cause conflicts.
- You start from a clean build state (important for reproducibility).

Maven Basics

clean phase

Example: In your automation project, it deletes:

```
/target/test-output/  
/target/classes/
```

- so that when you re-run mvn test, you get a **fresh, clean execution report**.

Maven Basics

compile phase

What it does:

- Compiles source code from **src/main/java** into bytecode (**.class** files) inside **/target/classes**.

In a real IT project:

- If it's a web app: this compiles your backend code (Java, Spring Boot).
- If it's an automation framework: it compiles Selenium utility classes, test base, and listener code.

Note: If the code fails to compile (like syntax errors or missing methods), the build stops here — before even running tests.

Example: In your automation project,

- If you have `BaseTest.java`, `WebDriverFactory.java`, and `LoginPage.java`, Maven compiles them here.

Maven Basics

test phase

What it does:

- Executes your test code from **src/test/java**.
- Uses TestNG or JUnit as defined in dependencies.
- Generates reports in **/target/surefire-reports/** or **/test-output/**.

In a real IT project:

- Developers or QA automation engineers execute: **mvn test**
- This runs all TestNG test cases (like Demoblaze login, add-to-cart).
- Generates HTML and XML reports.
- If integrated in Jenkins, this phase automatically publishes those reports in the pipeline dashboard.

Example: Runs your **testng.xml** suite for Demoblaze with all **@Test** annotations.

Maven Basics

package phase

What it does:

- Packages the compiled code into a deliverable artifact (e.g., .jar, .war).

In a real IT project:

- For a **web app**: builds a .war file to be deployed to Tomcat or a .jar for Spring Boot.
- For an **automation framework**: creates a .jar that can be executed by Jenkins or shared with other teams.

Example: Automation team packages the Selenium + TestNG project as:

Demoblaze_Automation-1.0.jar

- Then Jenkins uses this artifact in nightly regression jobs.

Maven Basics

verify phase

What it does:

- Runs integration tests, quality checks, and verifies that packages meet expected conditions before installation.

In a real IT project: This is where:

- Teams integrate SonarQube, Jacoco, or Checkstyle to check **code quality, test coverage, and security vulnerabilities**.
- Maven ensures your code passes all quality gates before it's approved.

Example: Automation team integrates:

- SonarQube for static code analysis.
- Jacoco plugin to ensure test coverage above 80%.

Maven Basics

install phase

What it does:

- Installs the packaged artifact (.jar or .war) into your local Maven repository (.m2 folder).

In a real IT project: This is where:

- Other microservices or modules can reuse this artifact without downloading again.
- Automation engineers can run the same jar across local machines.

Example: Once the framework passes all tests:

`~/m2/repository/com/kiot/fdp/DemoblazeAutomation/1.0/DemoblazeAutomation-1.0.jar`

- is saved locally, ready for Jenkins or other projects.

Maven Basics

deploy phase

What it does:

- Uploads the final build artifact to a remote repository (like Nexus, Artifactory, or GitHub Packages).

In a real IT project: This is where:

- After all checks pass, Jenkins pipeline automatically deploys artifacts to a central repository. DevOps or QA teams pull these artifacts into higher environments (QA, UAT, Prod).

Example: After packaging:

mvn deploy

- uploads your .jar or .war to: **<https://nexus.company.com/repository/releases/>**
- is an example of a Nexus Repository Manager endpoint — the place where Maven (and other build tools) uploads (“deploys”) built artifacts like JAR/WAR files after successful testing and packaging.
- Now other teams (CI, QA) can download and use the same tested build.

Real-time Example: CI/CD Flow (End-to-End)

Stage	Command	Activity	Tool
Build Validation	<code>mvn validate</code>	Checks POM, dependencies	Maven
Clean Setup	<code>mvn clean</code>	Removes old builds	Jenkins pre-step
Compilation	<code>mvn compile</code>	Compiles Selenium framework	Maven
Unit/Automation Tests	<code>mvn test</code>	Runs TestNG test cases	Jenkins
Packaging	<code>mvn package</code>	Creates <code>.jar</code> or <code>.war</code>	Maven
Code Quality Check	<code>mvn verify</code>	Runs SonarQube & coverage checks	SonarQube
Artifact Upload	<code>mvn install / deploy</code>	Publishes artifact to repo	Nexus/JFrog
Reporting	N/A	Jenkins publishes reports	Jenkins + Extent

Plugin Goals

- A **build phase** in Maven is responsible for a **specific step in the build lifecycle**.
- The way a **build phase performs its tasks** can vary **depending on its implementation**.
- This variation is achieved **by associating plugin goals** with the build phases.
- One **plugin goal** can be bound to none, one or more than one phase.
- You can call a goal that is not linked to a build phase by mvn dependency:copy-dependencies.

Plugin Goals

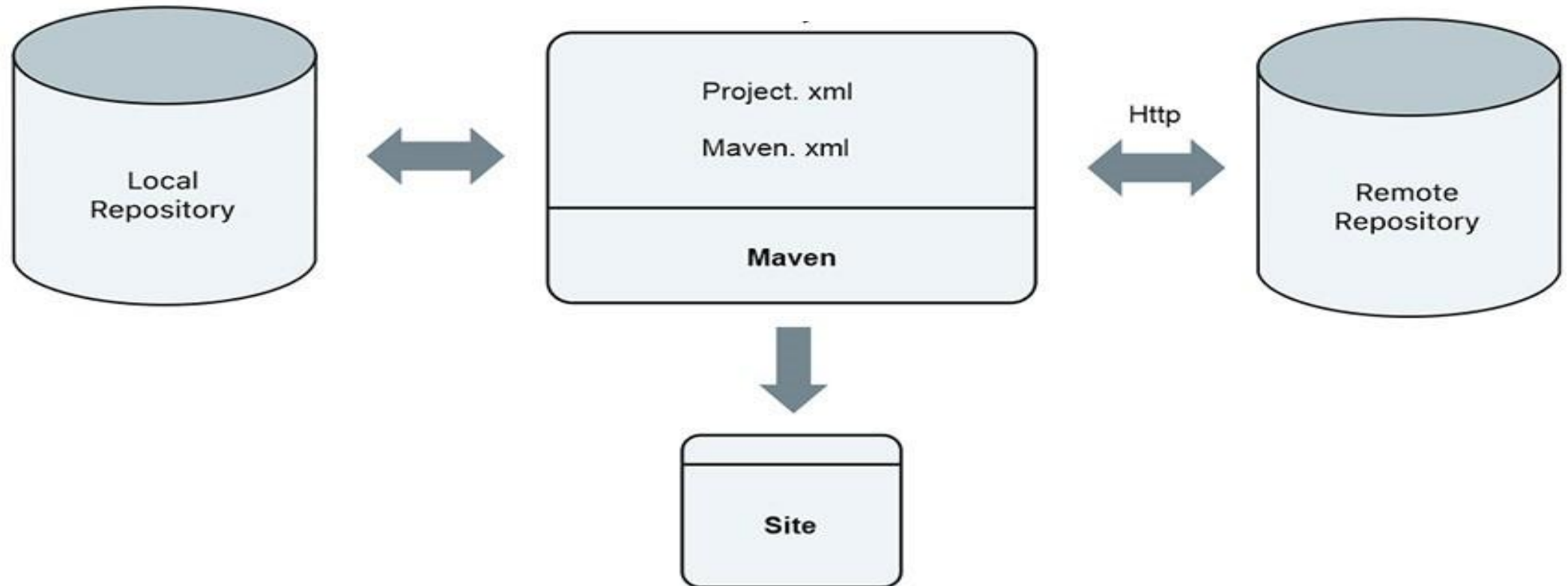
- Another way to **add goals** to phases is to **configure plugins** in your project.
- Plugins are **artifacts** that **provide goals to Maven**.
- A plugin may have **one or more goals** wherein each goal represents a capability of that plugin.
- For example, the Compiler plugin has two goals: **compile** and **testCompile**. The former compiles the source code of your main code, while the latter compiles the source code of your test code.
- There are basically
 - **Build plugins** are responsible for the build process (configurable in the **<build>** element of the POM)
 - **Reporting plugins** are responsible for the site generation (configurable in the **<reporting>** element of the POM)

Maven Repositories

- Maven searches the **repositories** for **dependencies in sequence**.
- First in the **local repository**, then in **remote repositories** if specified in the POM.
- The **local repository** is a **directory on the computer** where Maven runs. It caches remote downloads and contains temporary build artifacts that you have not yet released.
- The **remote repositories** refer to any other type of repository, accessed by a variety of protocols such as **file://** and **https://**.
- Other "**remote**" **repositories** may be internal repositories set up on a **file or HTTP server within your company**, used to share **private artifacts between development teams and for releases**.

Maven Basics

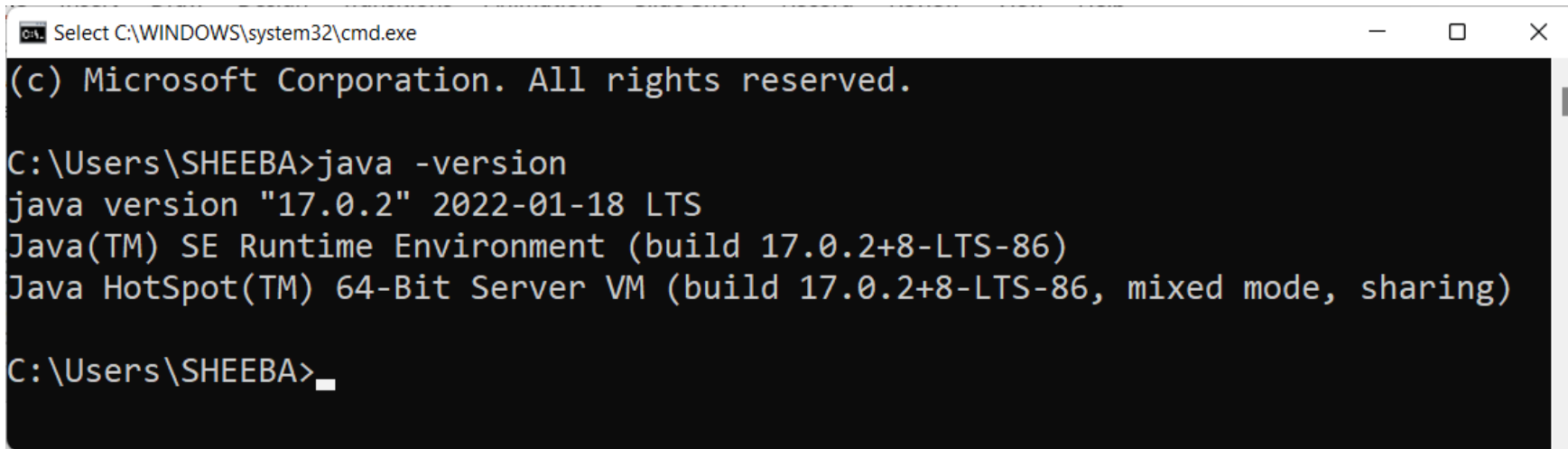
Maven Architecture



Environment Setup

- **Prerequisites**

- Maven is written in Java. So, to run Maven, we need a system that has Java installed and configured properly.
- To check, we will run the command below in CMD to get the currently installed version



```
Select C:\WINDOWS\system32\cmd.exe

(c) Microsoft Corporation. All rights reserved.

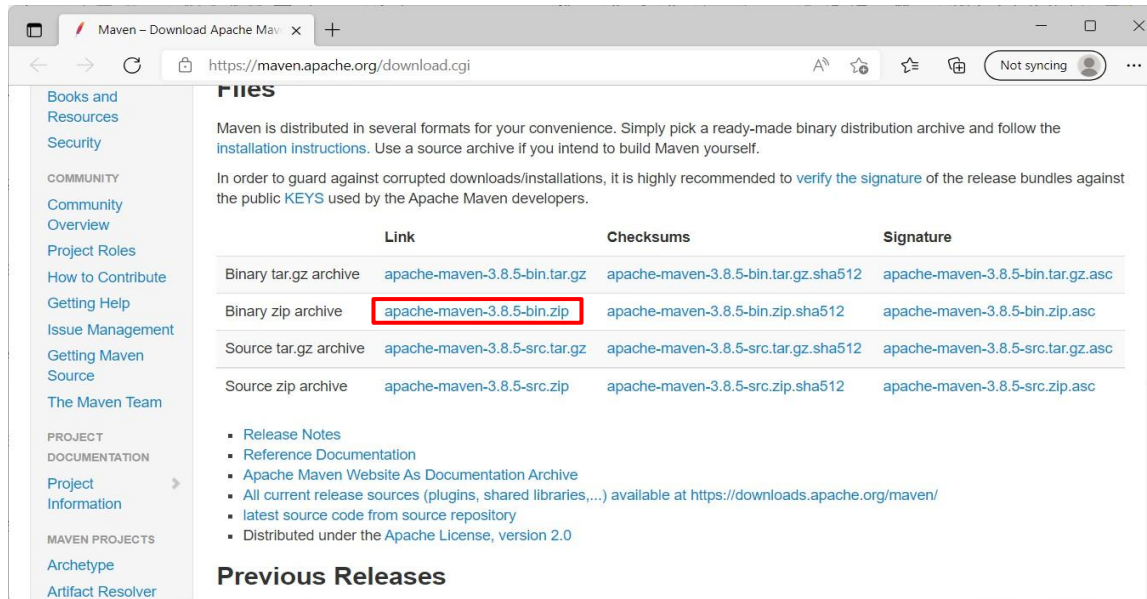
C:\Users\SHEEBA>java -version
java version "17.0.2" 2022-01-18 LTS
Java(TM) SE Runtime Environment (build 17.0.2+8-LTS-86)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.2+8-LTS-86, mixed mode, sharing)

C:\Users\SHEEBA>
```

Maven Basics

Environment Setup

- Download the Maven binary Archive from the link <https://maven.apache.org/download.cgi>



- Unzip the downloaded archive into the folder **C:\Program Files\Maven**. Copy the path to the clipboard.

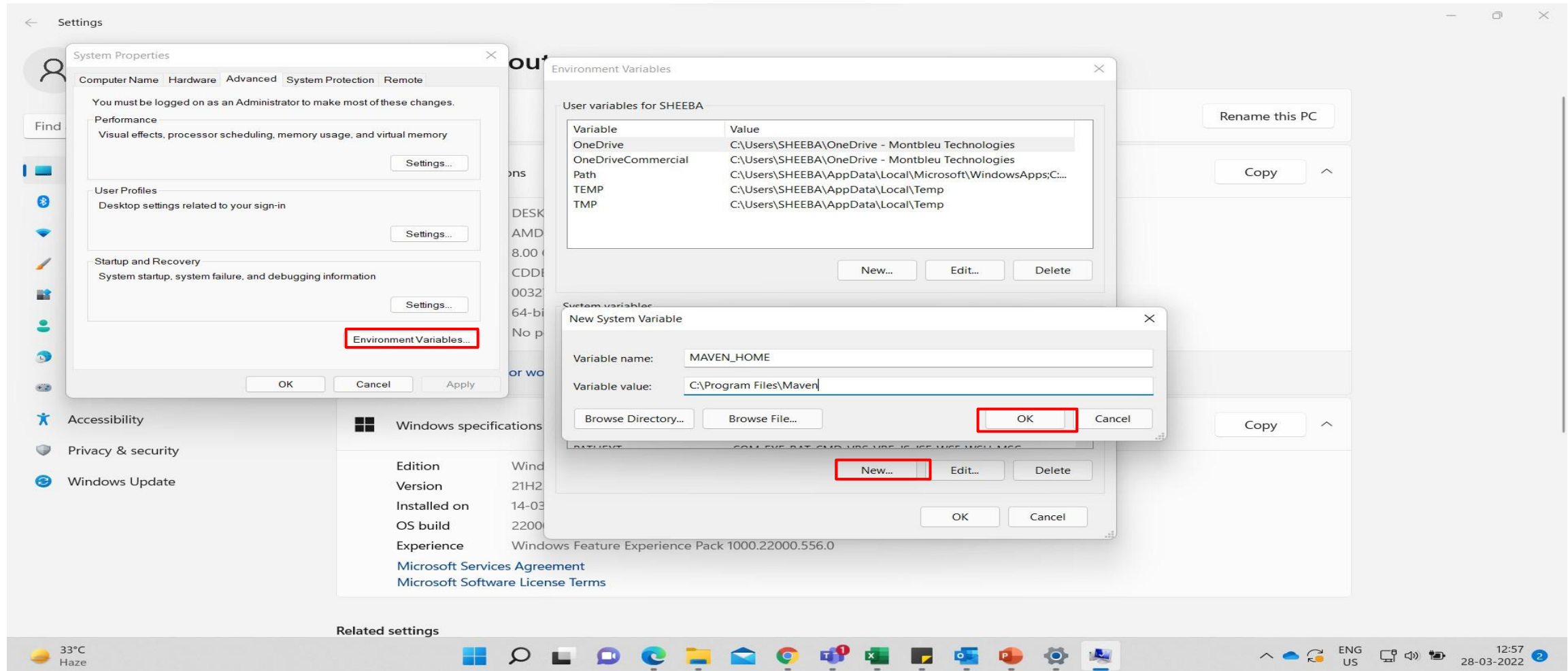
You can move the folder to any location of your choice, but it would be more convenient to keep it in the mentioned folder.

Environment Setup

Environment variable setting:

- Right-click on the Windows Start button and select System in the menu.
- In the new window, click on the 'Advanced System Settings'.
- Click on the 'Environment Variables' in the pop-up window.
- Click on 'New' in the new pop-up window in the System variables.
- Enter MAVEN_HOME (all characters in capital case) in the Variable name field.
- Paste the copied earlier path to the Maven folder in the Variable value. Click on the OK button.

Environment Setup



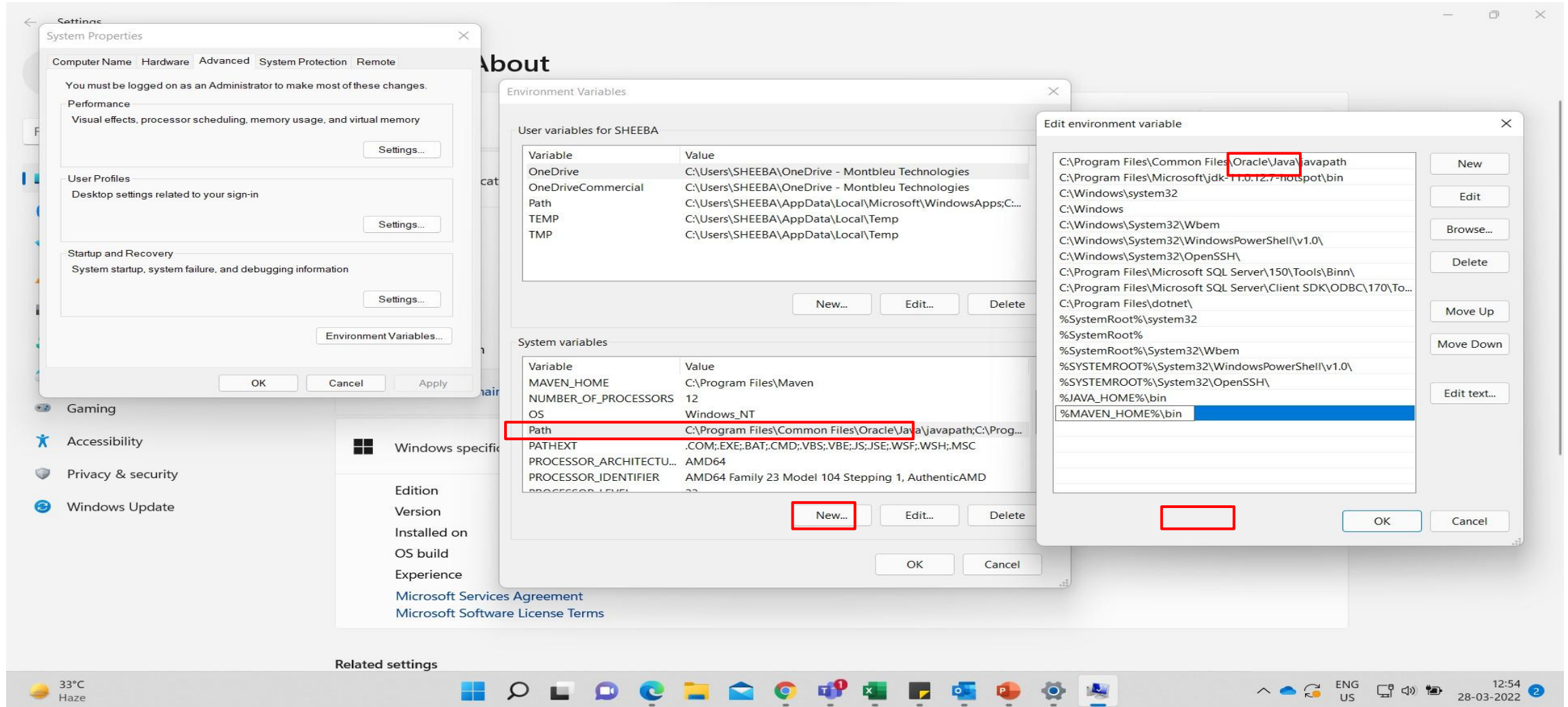
Environment Setup

Environment variable setting:

- Select the Path variable and click on Edit.
- In the new window, click on New.
- In the new field, enter %MAVEN_HOME%\bin.
- After that, click OK in all the previously opened windows.

Maven Basics

Environment Setup



The screenshot illustrates the steps to configure the environment for Maven on a Windows system. It shows three overlapping windows:

- System Properties:** The 'Advanced' tab is selected, and the 'Environment Variables...' button is visible at the bottom.
- Environment Variables:** This window shows user and system variables. The 'Path' variable under 'System variables' is highlighted with a red box. Below it, the 'New...' button is also highlighted with a red box.
- Edit environment variable:** This window shows the 'Path' variable being edited. The path 'C:\Program Files\Common Files\Oracle\Java\javapath' is highlighted with a red box. At the bottom, an empty text input field is highlighted with a red box, indicating where to add the new path.

The taskbar at the bottom shows the system clock as 12:54 on 28-03-2022, and the language is set to ENG US.

Environment Setup

- To verify the proper installation of Maven, we will run the command below in CMD to get the currently installed version



```
Microsoft Windows [Version 10.0.22000.556]
(c) Microsoft Corporation. All rights reserved.

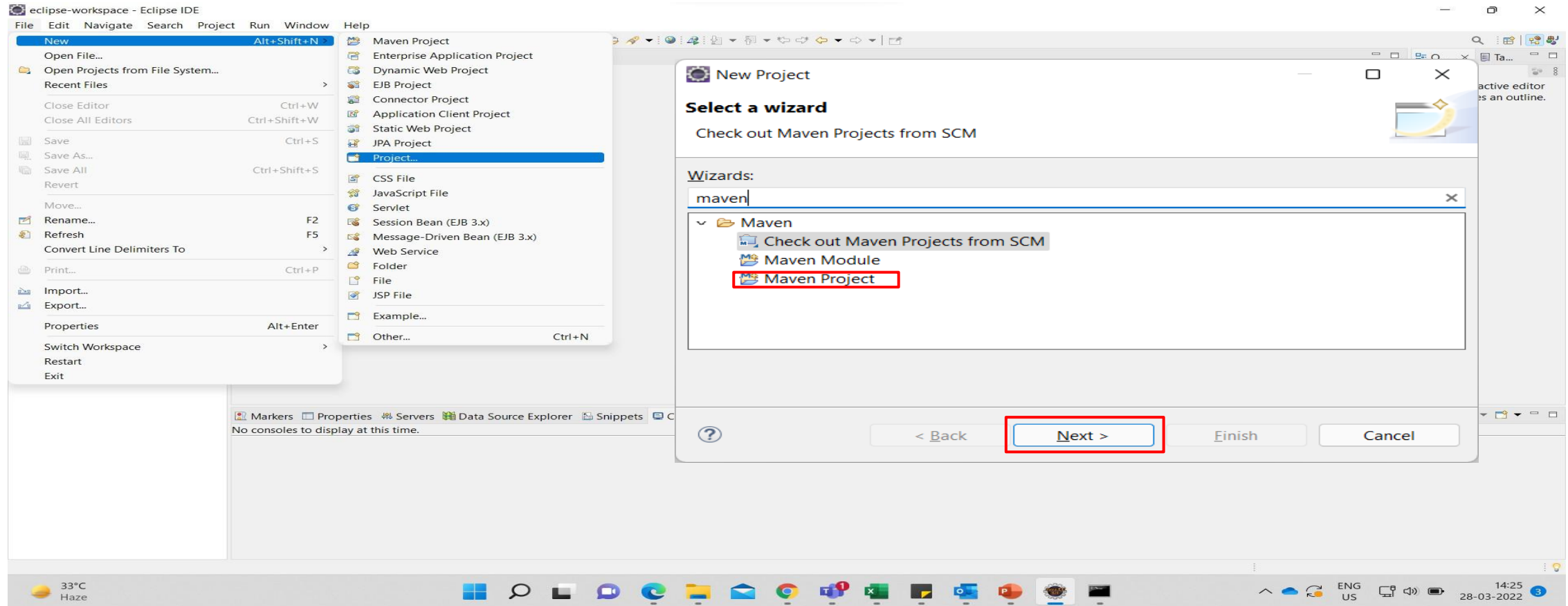
C:\Users\SHEEBA>mvn -version
Apache Maven 3.8.5 (3599d3414f046de2324203b78ddcf9b5e4388aa0)
Maven home: C:\Program Files\Maven
Java version: 17.0.2, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17.0.2
Default locale: en_IN, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

C:\Users\SHEEBA>
```

Maven Basics

Demo - Using Maven with Eclipse IDE

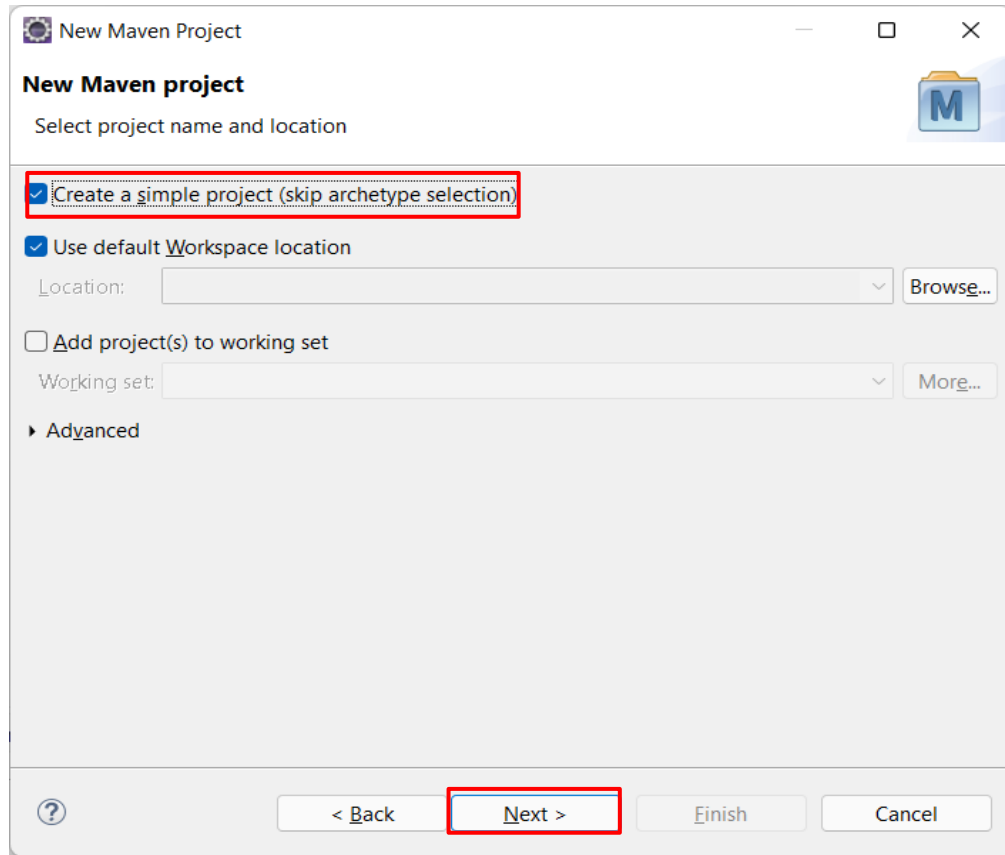
Create the new project -> Maven Project



Maven Basics

Demo - Using Maven with Eclipse IDE

Change/Provide the required configuration details



New Maven Project

Select project name and location

☒ Create a simple project (skip archetype selection)

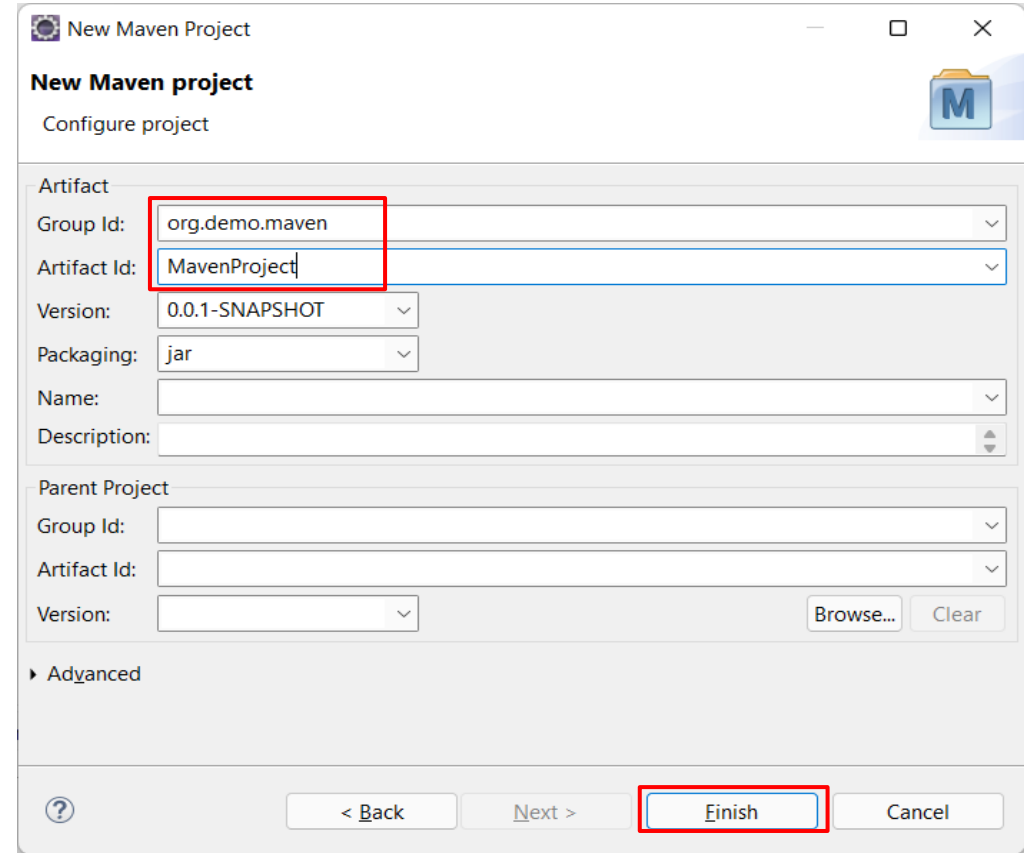
☒ Use default Workspace location

Location:

☐ Add project(s) to working set

Working set:

► Advanced



New Maven Project

Configure project

Artifact

Group Id:

Artifact Id:

Version:

Packaging:

Name:

Description:

Parent Project

Group Id:

Artifact Id:

Version:

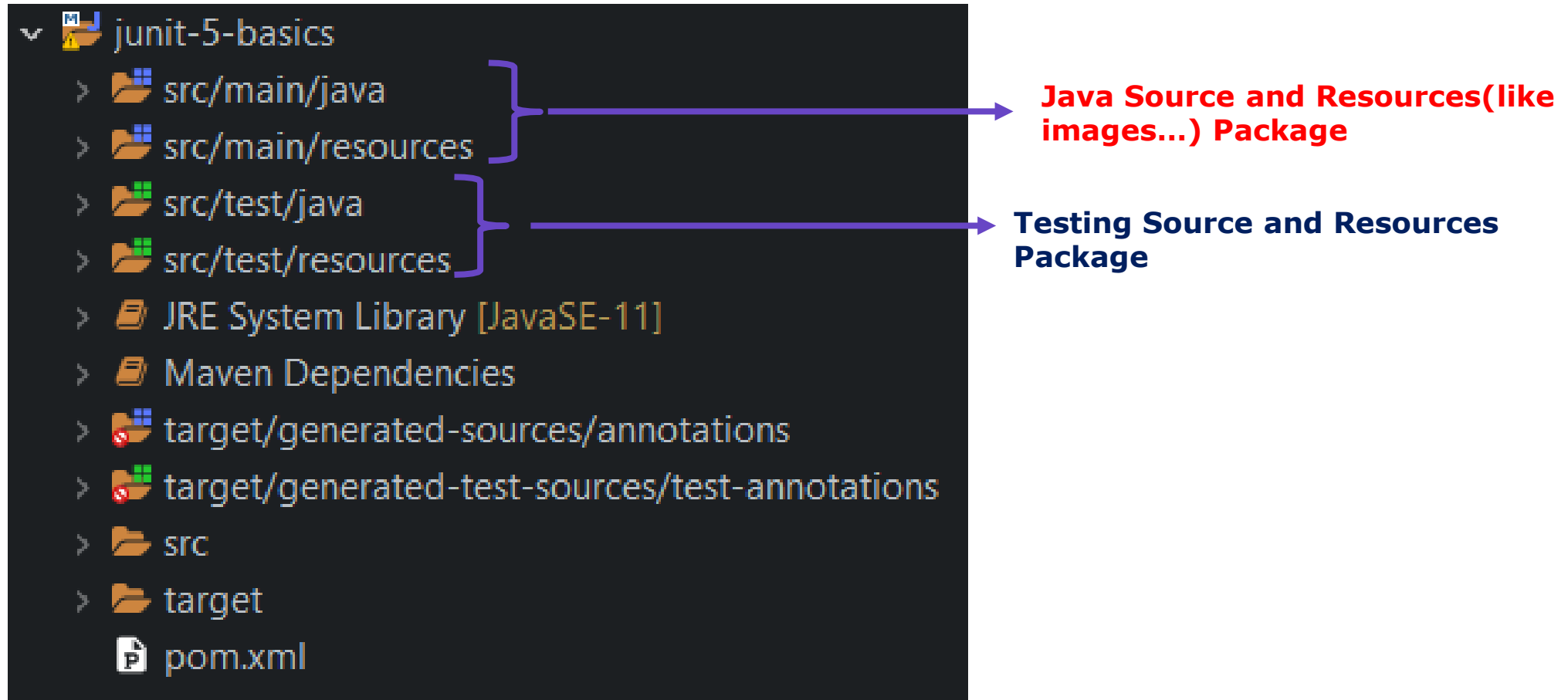
► Advanced

Guide to naming conventions on groupId, artifactId, and version

- **groupId** uniquely **identifies** your **project across all projects**. A group ID should follow Java's package name rules. This means it starts with a reversed domain name you control.
 - For **example**, org.apache.maven, org.apache.commons
- **artifactId** is the **name of the jar without version**. If you created it, then you can choose whatever name you want with lowercase letters and no strange symbols. If it's a third party jar, you must take the name of the jar as it's distributed.
 - **Example**: maven, commons-math
- version if you distribute it, then you can choose any typical version with numbers and **dots (1.0, 1.1, 1.0.1, ...)**. Don't use dates .
- For more details look: <https://maven.apache.org/guides/mini/guide-naming-conventions.html>

Maven Basics

Demo – Understand the Project Structure



Project Object Model(POM)

Configuration of a Maven project is done via **pom.xml** configuration file.

The POM file defines:

- **identifiers** for the project to be build
- **properties** relevant for **build configuration**
- **plugins** which provide functionality for **the build via a build section.**
- **library and project dependencies** via the dependencies section

Maven Basics

pom.xml



The screenshot displays an IDE with a Maven project named 'MavenDemo'. On the left, the 'Project Explorer' shows the project structure, including 'src/main/java', 'src/test/java', 'JRE System Library [J2SE-1.5]', 'Maven Dependencies' (circled in red), 'src', 'target', and 'pom.xml'. On the right, the 'pom.xml' file is open, showing the following XML content:

```

1 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://
2   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven
3   <modelVersion>4.0.0</modelVersion>
4
5   <groupId>training</groupId>
6   <artifactId>MavenDemo</artifactId>
7   <version>0.0.1-SNAPSHOT</version>
8   <packaging>jar</packaging>
9
10  <name>MavenDemo</name>
11  <url>http://maven.apache.org</url>
12
13  <properties>
14    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15  </properties>
16
17  <dependencies>
18    <dependency>
19      <groupId>junit</groupId>
20      <artifactId>junit</artifactId>
21      <version>3.8.1</version>
22      <scope>test</scope>
23    </dependency>
24  </dependencies>
  
```

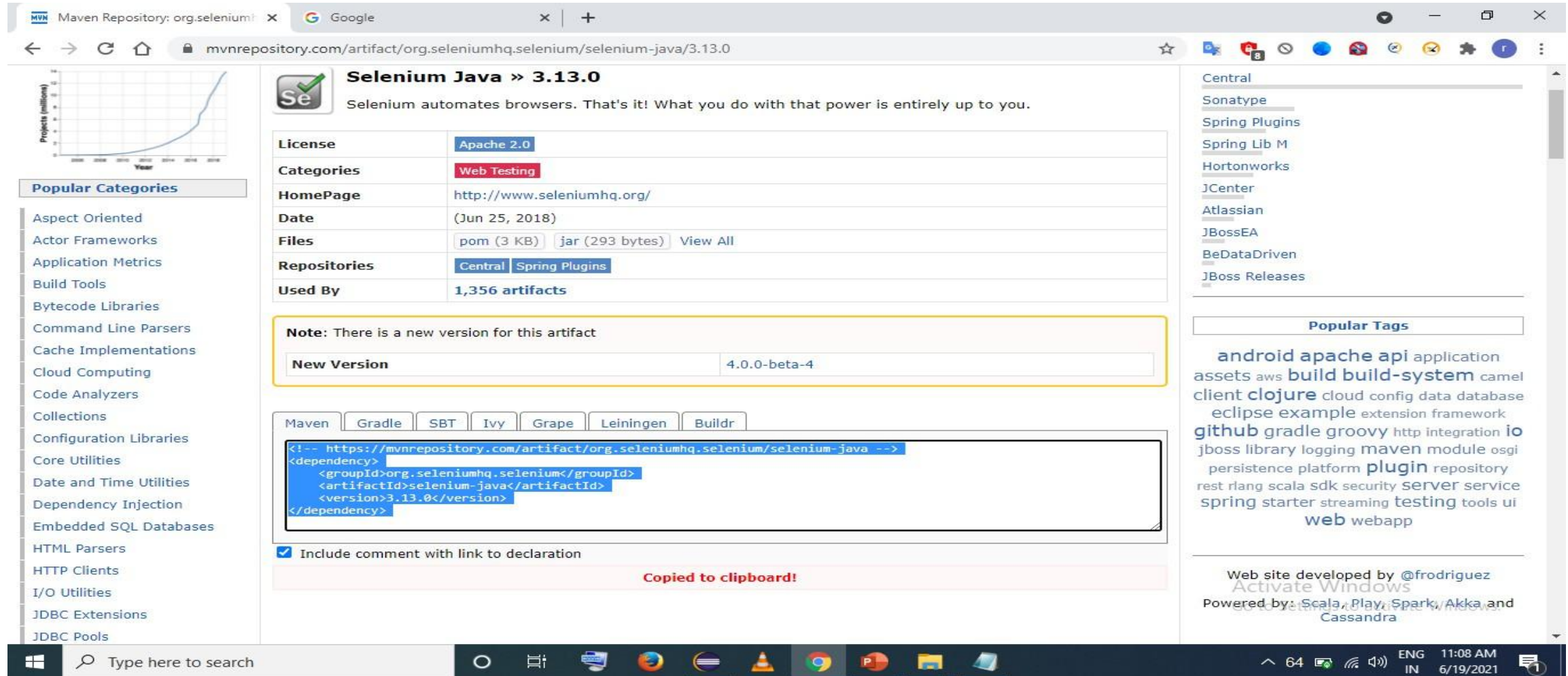
The dependency block (lines 17-24) is circled in red, matching the 'Maven Dependencies' folder in the Project Explorer.

Dependencies

- Get it from: <https://mvnrepository.com/>
- In search box -----→search selenium---→ under selenium java--→3.13.0(Any stable version)
- In search box -----→search testing--→ under testing-→6.14.3(Any stable version)

Maven Basics

Getting Selenium Java dependency



Selenium Java » 3.13.0
 Selenium automates browsers. That's it! What you do with that power is entirely up to you.

License	Apache 2.0
Categories	Web Testing
HomePage	http://www.seleniumhq.org/
Date	(Jun 25, 2018)
Files	pom (3 KB) jar (293 bytes) View All
Repositories	Central Spring Plugins
Used By	1,356 artifacts

Note: There is a new version for this artifact

New Version [4.0.0-beta-4](#)

Maven [Gradle](#) [SBT](#) [Ivy](#) [Grape](#) [Leiningen](#) [Buildr](#)

```
<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>3.13.0</version>
</dependency>
```

☒ Include comment with link to declaration

Copied to clipboard!

Popular Categories

- Aspect Oriented
- Actor Frameworks
- Application Metrics
- Build Tools
- Bytecode Libraries
- Command Line Parsers
- Cache Implementations
- Cloud Computing
- Code Analyzers
- Collections
- Configuration Libraries
- Core Utilities
- Date and Time Utilities
- Dependency Injection
- Embedded SQL Databases
- HTML Parsers
- HTTP Clients
- I/O Utilities
- JDBC Extensions
- JDBC Pools

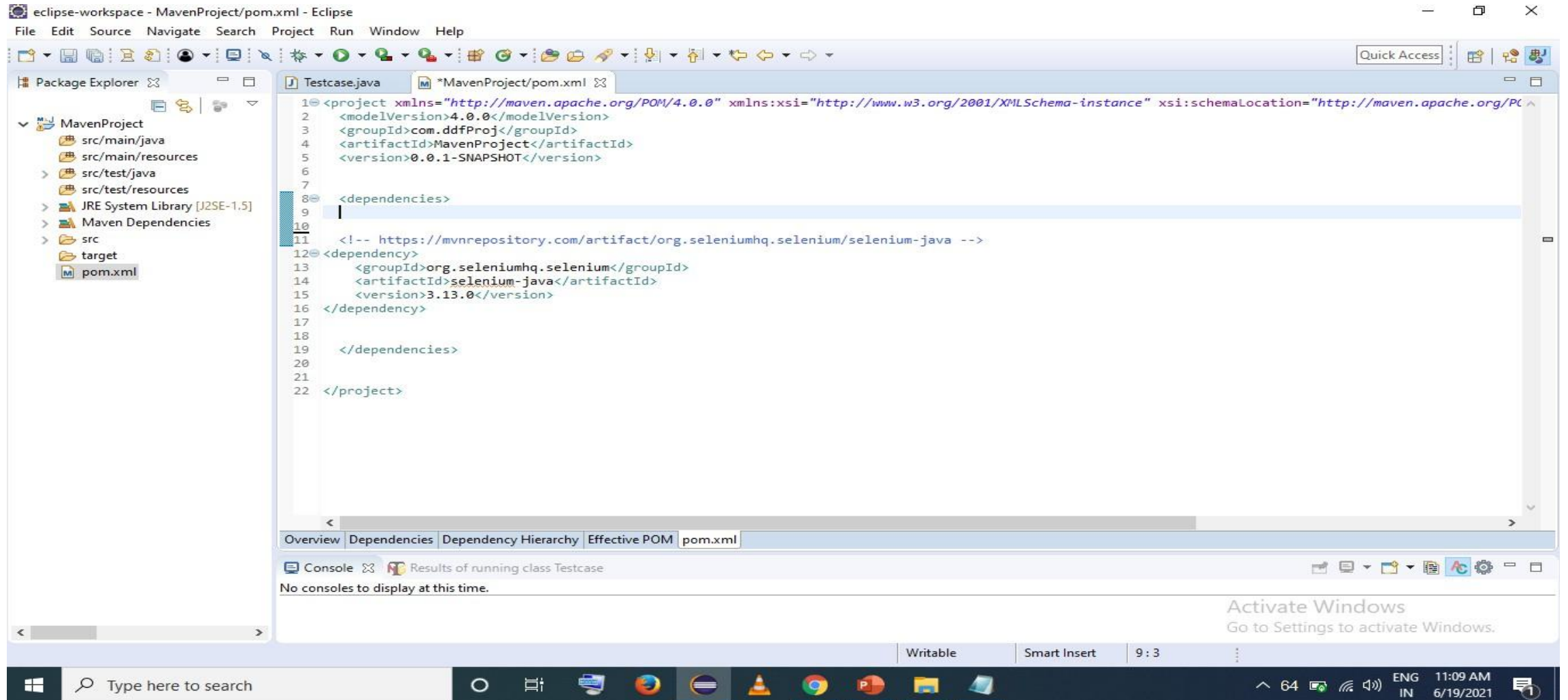
Popular Tags

android apache api application assets aws build build-system camel client clojure cloud config data database eclipse example extension framework github gradle groovy http integration io jboss library logging maven module osgi persistence platform plugin repository rest rang scala sdk security server service spring starter streaming testing tools ui web webapp

Web site developed by @frodriquez
 Powered by: Scala, Play, Spark, Akka and Cassandra

Maven Basics

Paste dependencies in pom.xml



Plugins

- Maven is a **plugin execution framework** where every task is actually done by plugins.
- Maven Plugins are generally used to
 - create jar file
 - create war file
 - compile code files
 - unit testing of code
 - create project documentation
 - create project reports

Maven Basics

Plugin

Surefire plugin

<https://maven.apache.org/surefire/maven-surefire-plugin/usage.html>

Clean plugin

<https://maven.apache.org/plugins/maven-clean-plugin/plugin-info.html>

Compiler plugin

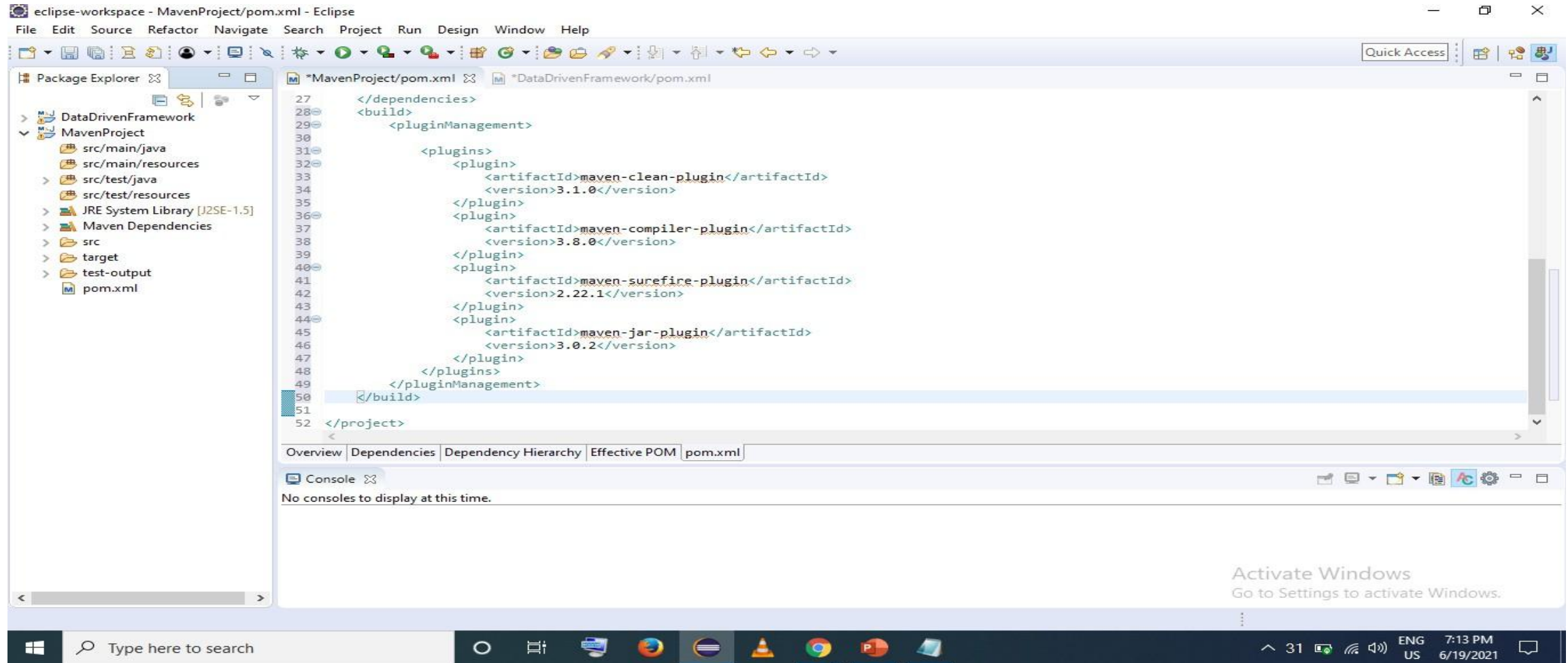
<https://maven.apache.org/plugins/maven-compiler-plugin/plugin-info.html>

Jar plugin

<https://maven.apache.org/plugins/maven-jar-plugin/plugin-info.html>

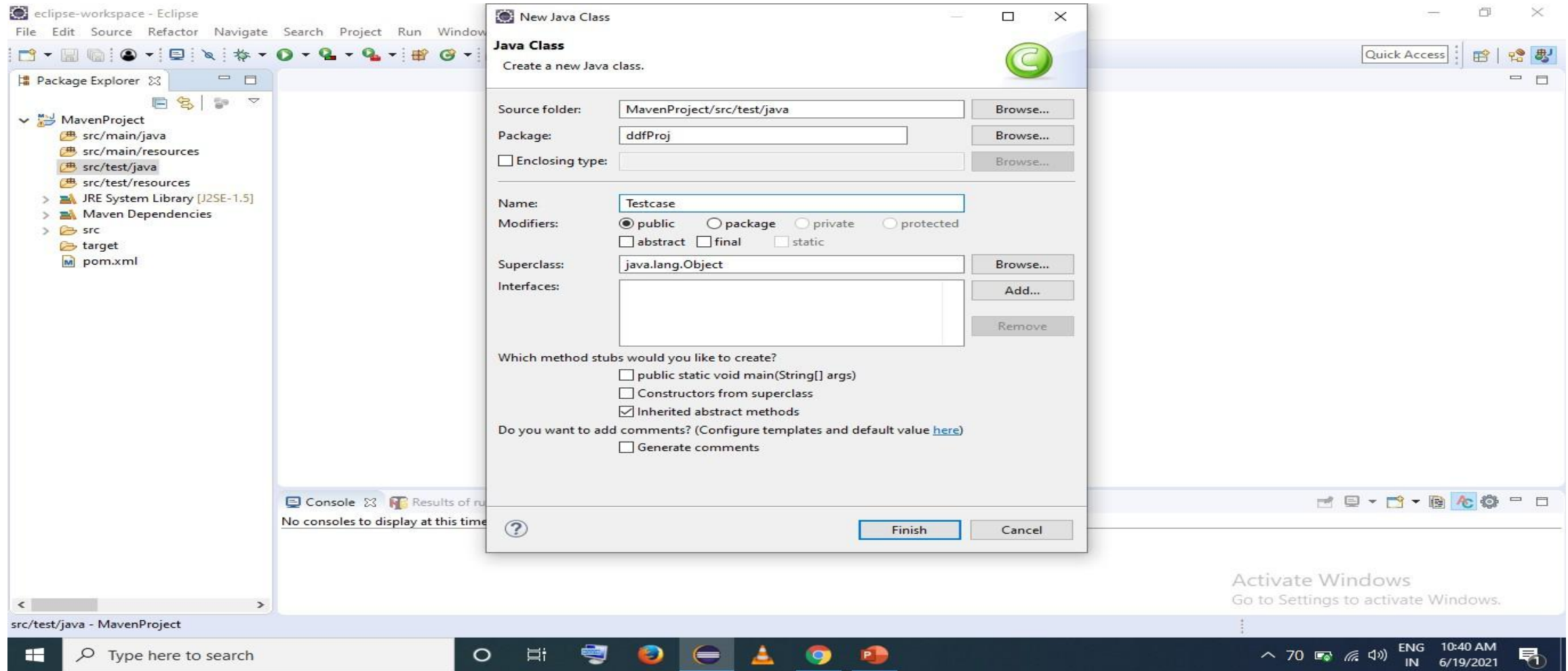
Maven Basics

Adding Plugins



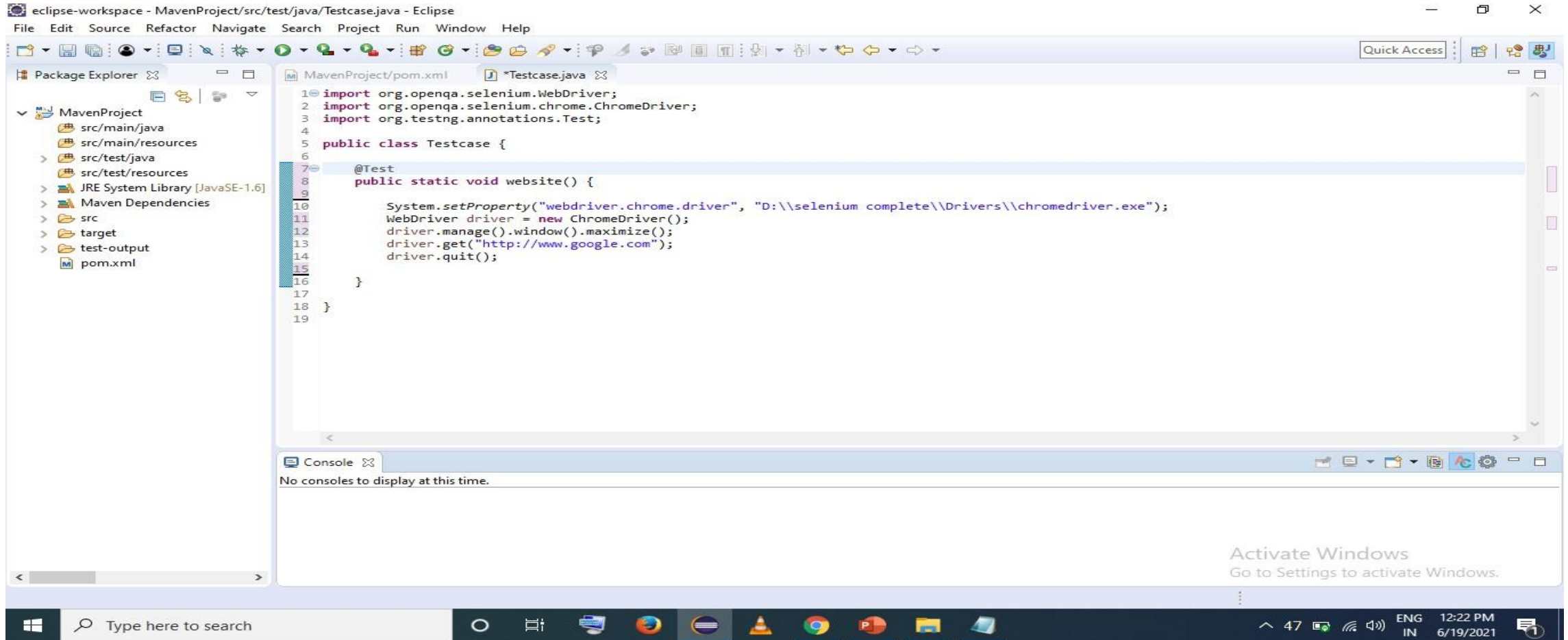
Maven Basics

Creating Testcase



Maven Basics

Creating Testcase



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: MavenProject, src/main/java, src/main/resources, src/test/java, src/test/resources, JRE System Library [JavaSE-1.6], Maven Dependencies, src, target, test-output, and pom.xml. The main editor window shows the file MavenProject/pom.xml and *Testcase.java. The code in Testcase.java is as follows:

```

1 import org.openqa.selenium.WebDriver;
2 import org.openqa.selenium.chrome.ChromeDriver;
3 import org.testng.annotations.Test;
4
5 public class Testcase {
6
7     @Test
8     public static void website() {
9
10        System.setProperty("webdriver.chrome.driver", "D:\\selenium complete\\Drivers\\chromedriver.exe");
11        WebDriver driver = new ChromeDriver();
12        driver.manage().window().maximize();
13        driver.get("http://www.google.com");
14        driver.quit();
15    }
16
17 }
18
19

```

The Console window at the bottom shows the message: "No consoles to display at this time."

At the bottom right of the IDE, there is a watermark that says "Activate Windows Go to Settings to activate Windows."

Maven Basics

Maven Build

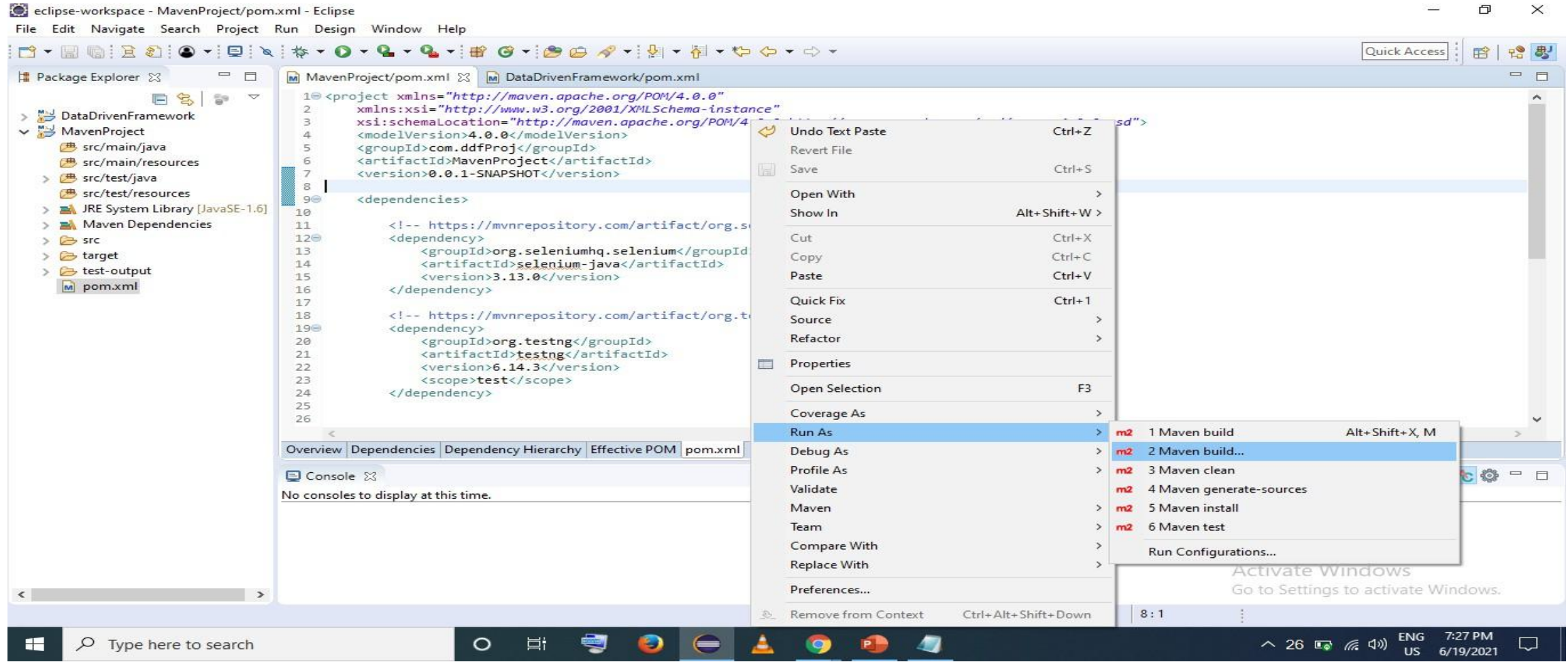
Using Maven Build we can use the following command:

- Clean
- Compile
- Test

And click -----run

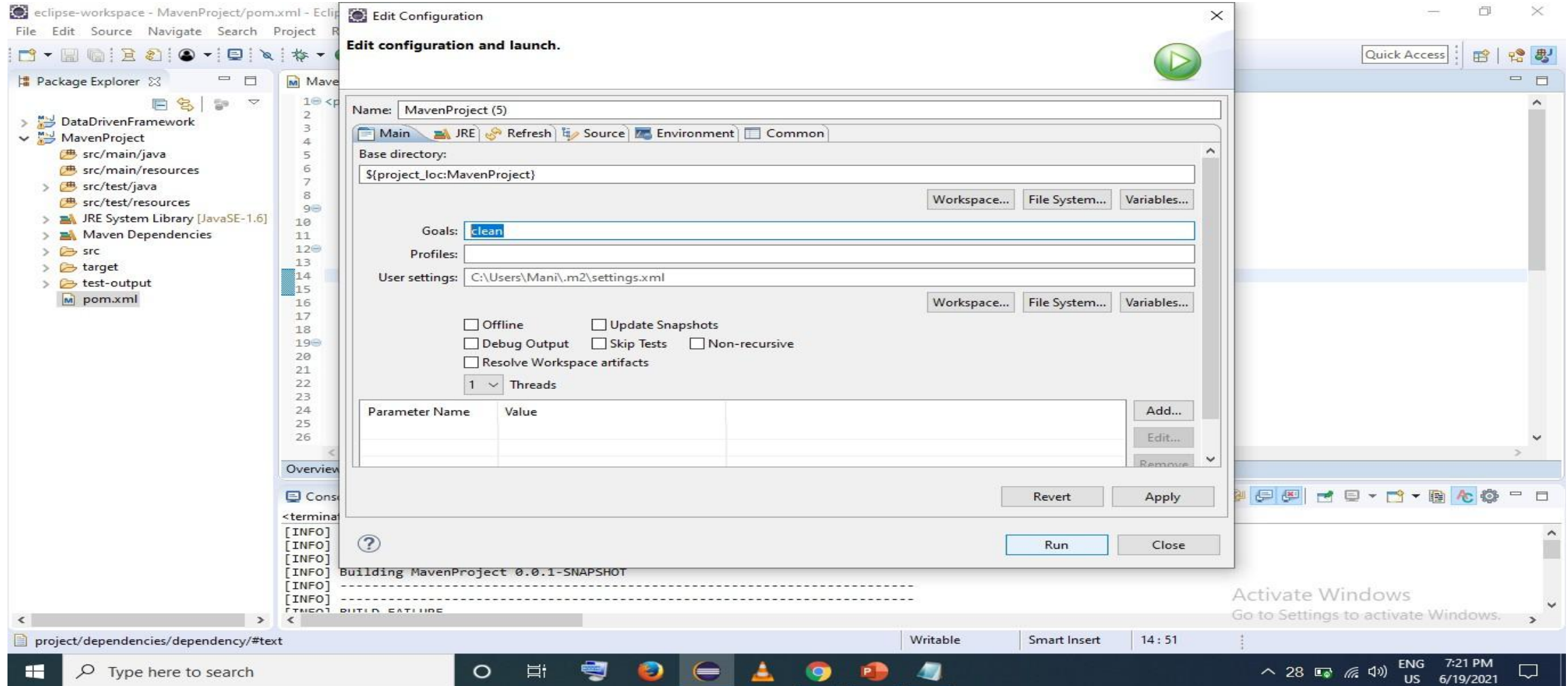
Maven Basics

Executing Maven Build



Maven Basics

Launch Project



Edit Configuration

Edit configuration and launch.

Name: MavenProject (5)

Base directory: \${project_loc:MavenProject}

Goals: clean

Profiles:

User settings: C:\Users\Mani\.m2\settings.xml

☐ Offline ☐ Update Snapshots

☐ Debug Output ☐ Skip Tests ☐ Non-recursive

☐ Resolve Workspace artifacts

1 Threads

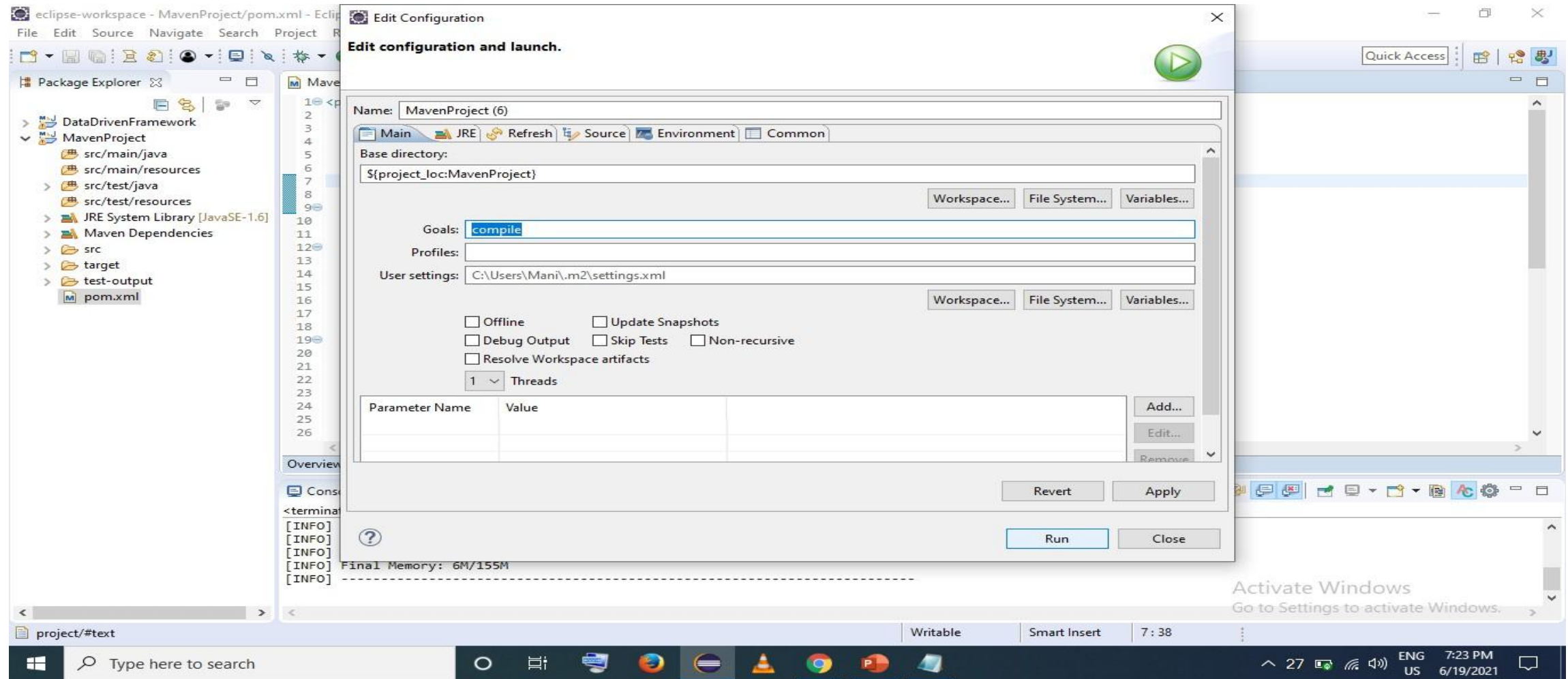
Parameter Name	Value

Revert Apply Run Close

Building MavenProject 0.0.1-SNAPSHOT

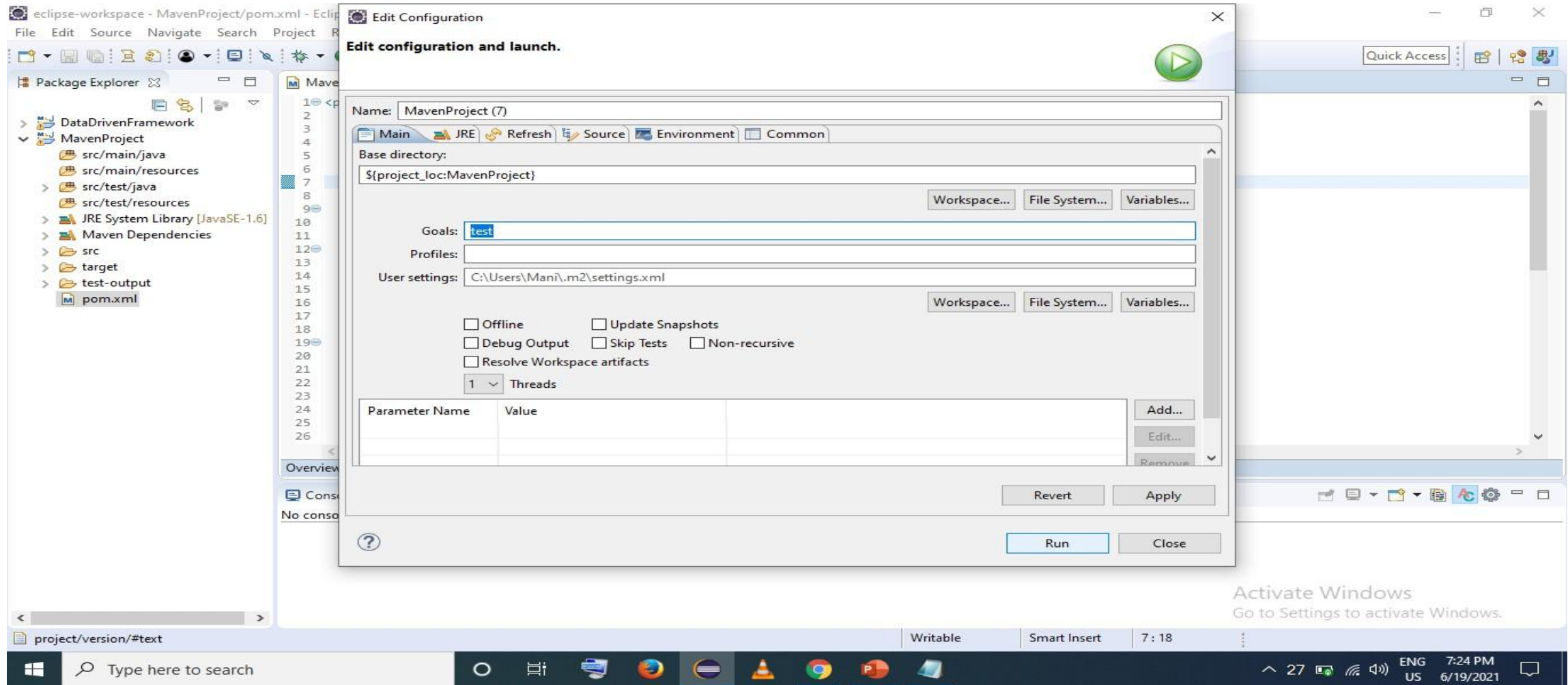
Maven Basics

Launch Project



Maven Basics

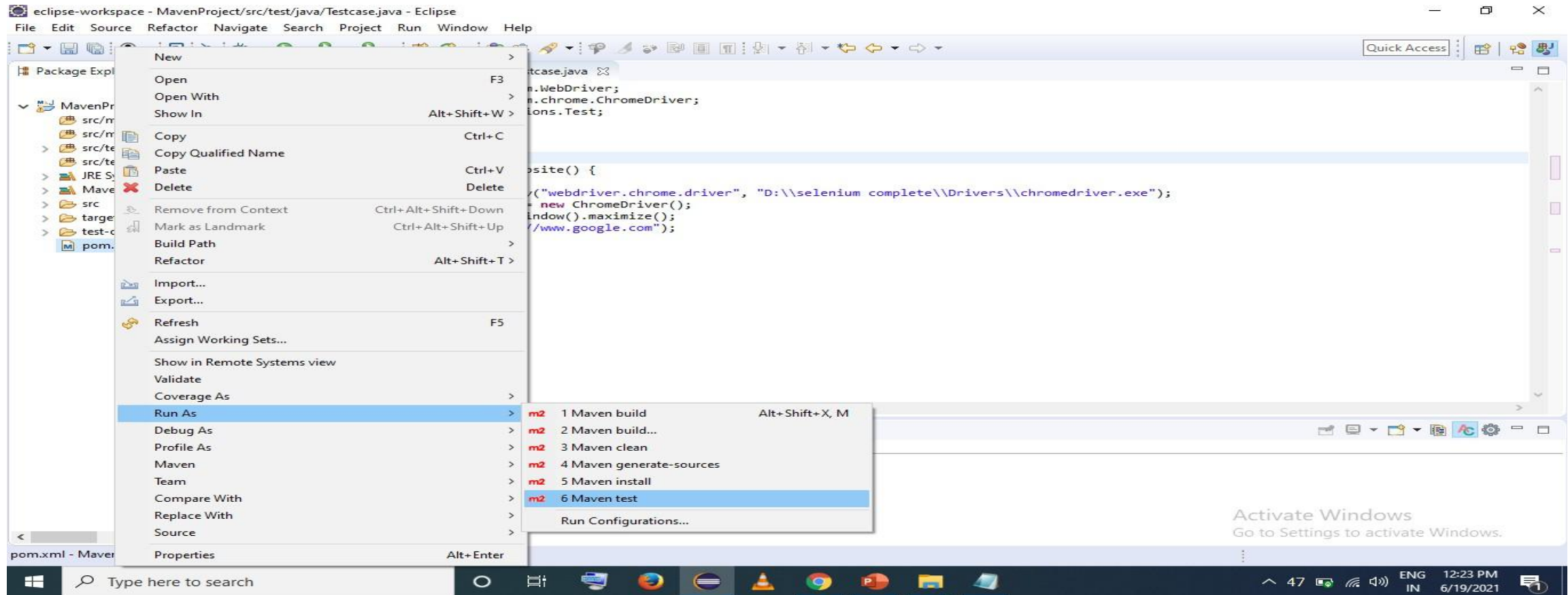
Test



Maven Basics

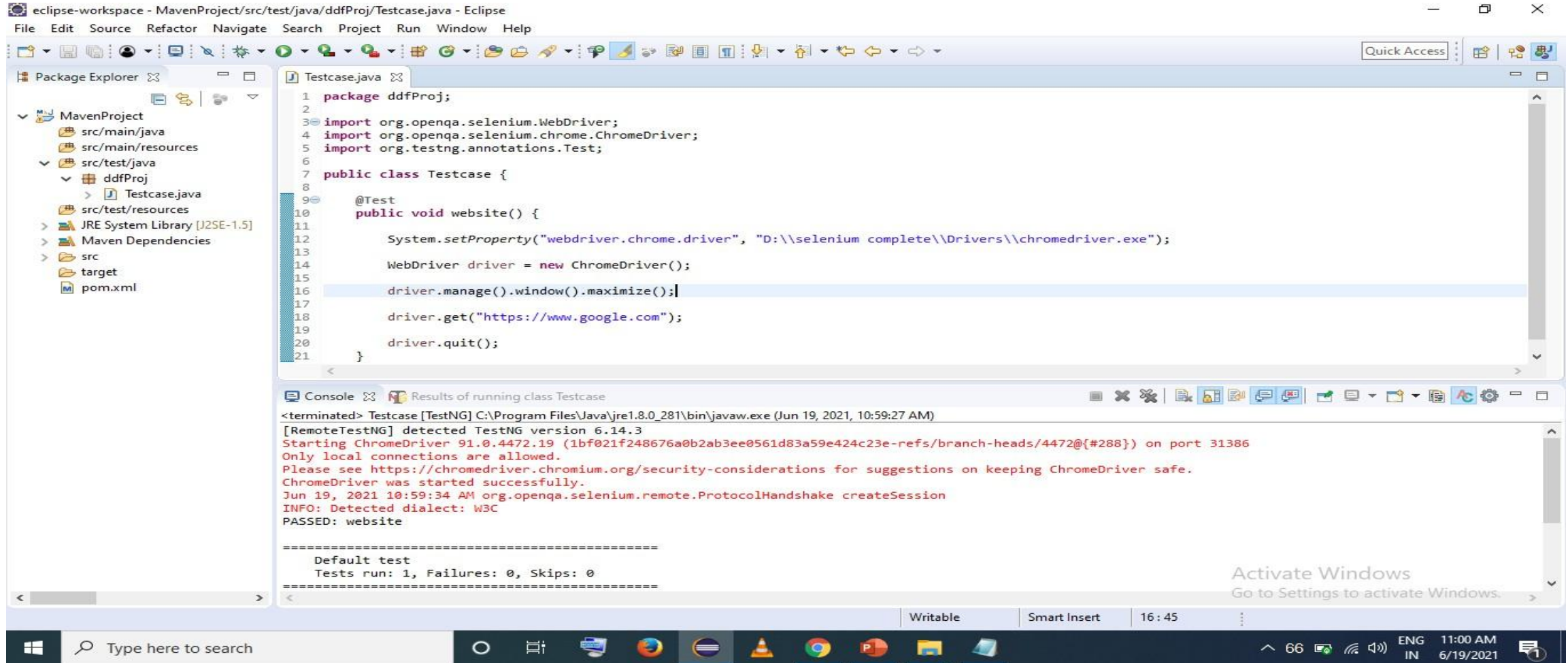
Without build command

Directly click clean and test by right clicking on pom.xml without using build



Maven Basics

Without build command



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: MavenProject > src/test/java > ddfProj > Testcase.java. The main editor shows the content of Testcase.java, which is a JUnit 4 test class using Selenium WebDriver to navigate to Google. The Console at the bottom shows the output of running the test, indicating a successful execution.

```

1 package ddfProj;
2
3 import org.openqa.selenium.WebDriver;
4 import org.openqa.selenium.chrome.ChromeDriver;
5 import org.testng.annotations.Test;
6
7 public class Testcase {
8
9     @Test
10    public void website() {
11
12        System.setProperty("webdriver.chrome.driver", "D:\\selenium complete\\Drivers\\chromedriver.exe");
13
14        WebDriver driver = new ChromeDriver();
15
16        driver.manage().window().maximize();
17
18        driver.get("https://www.google.com");
19
20        driver.quit();
21    }
22 }

```

```

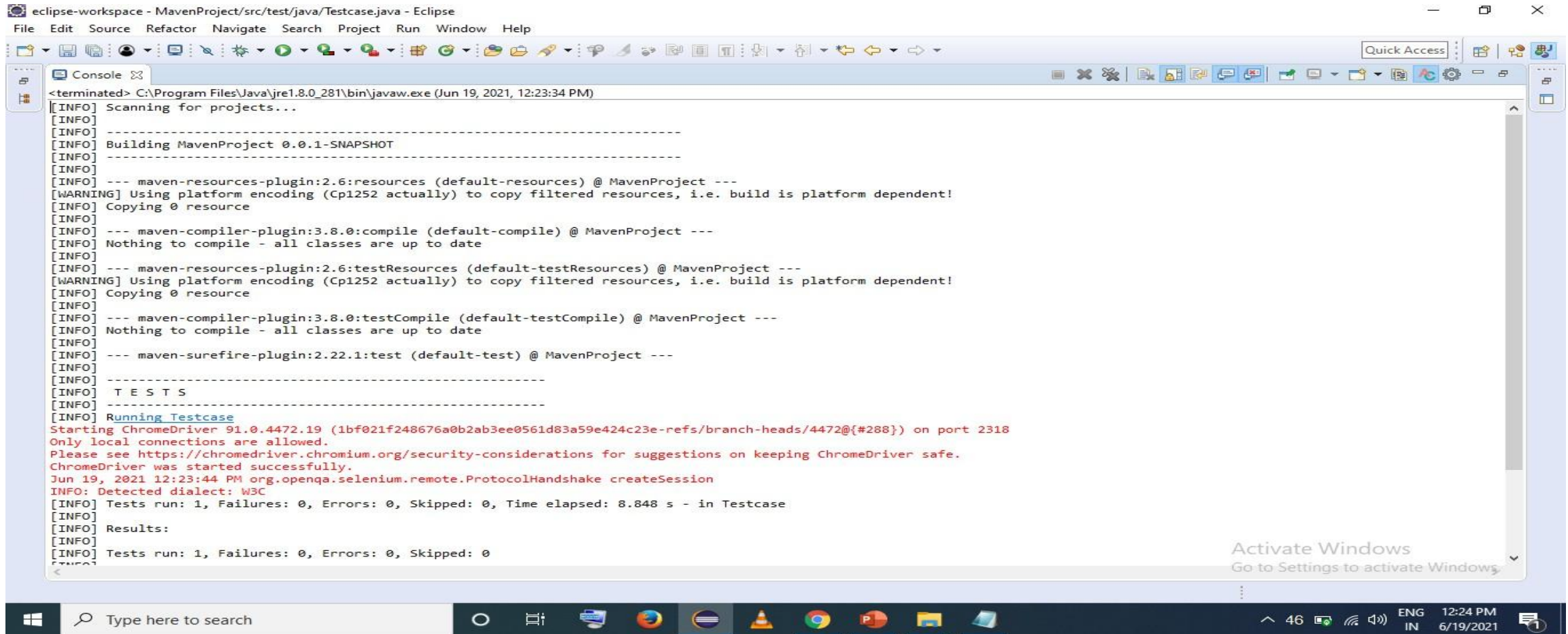
<terminated> Testcase [TestNG] C:\Program Files\Java\jre1.8.0_281\bin\javaw.exe (Jun 19, 2021, 10:59:27 AM)
[RemoteTestNG] detected TestNG version 6.14.3
Starting ChromeDriver 91.0.4472.19 (1bf021f248676a0b2ab3ee0561d83a59e424c23e-refs/branch-heads/4472@{#288}) on port 31386
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully.
Jun 19, 2021 10:59:34 AM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
PASSED: website

=====
Default test
Tests run: 1, Failures: 0, Skips: 0
=====

```

Maven Basics

Without build command



```

eclipse-workspace - MavenProject/src/test/java/Testcase.java - Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help

<terminated> C:\Program Files\Java\jre1.8.0_281\bin\javaw.exe (Jun 19, 2021, 12:23:34 PM)
[INFO] Scanning for projects...
[INFO]
[INFO] -----
[INFO] Building MavenProject 0.0.1-SNAPSHOT
[INFO] -----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MavenProject ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.8.0:compile (default-compile) @ MavenProject ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MavenProject ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.8.0:testCompile (default-testCompile) @ MavenProject ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.22.1:test (default-test) @ MavenProject ---
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running Testcase
Starting ChromeDriver 91.0.4472.19 (1bf021f248676a0b2ab3ee0561d83a59e424c23e-refs/branch-heads/4472@{#288}) on port 2318
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully.
Jun 19, 2021 12:23:44 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.848 s - in Testcase
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

```

Creating Maven-based Selenium + TestNG Project

- A Maven-based project is a structured Java project that uses Apache Maven to manage:
 - Dependencies (Selenium, TestNG, reporting libs, etc.)
 - Build lifecycle (compile, test, package)
 - Project structure
 - Execution through standardized commands (mvn test, mvn clean install)
- So instead of manually adding JARs, Maven downloads everything automatically from the Maven Central Repository using entries in a single file — pom.xml.

Creating Maven-based Selenium + TestNG Project

CREATING A MAVEN-BASED SELENIUM + TESTNG PROJECT



Maven

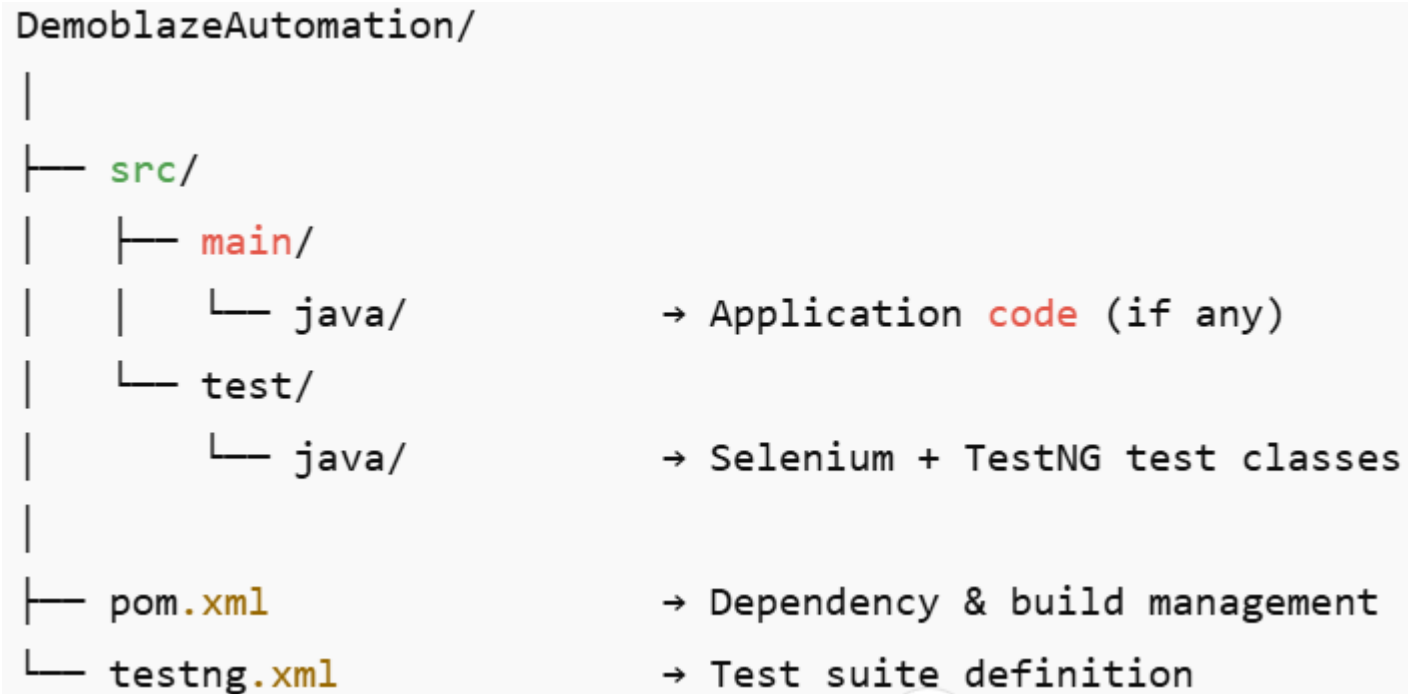
Build Lifecycle

Why Use Maven with Selenium + TestNG?

Problem (Without Maven)	Solution (With Maven)
Manual JAR management	Maven auto-downloads dependencies
No version control for libraries	Each dependency version is locked in <code>pom.xml</code>
Difficult CI/CD integration	Maven integrates easily with Jenkins
No standardized structure	Maven enforces a clean, modular structure
Hard to share project	One click: clone + build + run via <code>mvn test</code>

Step-by-Step: Creating a Maven + Selenium + TestNG Project

- Project Folder Structure



Step-by-Step: Creating a Maven + Selenium + TestNG Project

Step 1: Create a new Maven project

- In Eclipse:

```
File → New → Maven Project → Create a simple project  
Group Id: com.kiot.fdp  
Artifact Id: DemoblazeAutomation  
Version: 1.0
```

Eclipse auto-generates a `pom.xml` file.

Step-by-Step: Creating a Maven + Selenium + TestNG Project

Step 2: Add Selenium and TestNG Dependencies

- **Open pom.xml → inside <dependencies> tag, add:**

```
<dependencies>
  <!-- Selenium WebDriver -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.21.0</version>
  </dependency>
  <!-- TestNG Framework -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Maven will **automatically download all required JARs** from the **Maven Central Repository** into your local .m2 cache.

Step-by-Step: Creating a Maven + Selenium + TestNG Project

Step 3: Add the Maven Compiler Plugin (Optional but recommended)

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Step-by-Step: Creating a Maven + Selenium + TestNG Project

Step 4: Create Your First TestNG Test Under src/test/java, create: package tests; and Under that create Test class file

```
package tests;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.Test;
import io.github.bonigarcia.wdm.WebDriverManager;

public class VerifyHomePageTitle {

    @Test
    public void validateHomePageTitle() {
        WebDriverManager.chromedriver().setup();
        WebDriver driver = new ChromeDriver();
        driver.get("https://demoblaze.com/");
        String title = driver.getTitle();
        System.out.println("Page Title: " + title);
        Assert.assertEquals(title, "STORE", "Title mismatch!");
        driver.quit();
    }
}
```

Step-by-Step: Creating a Maven + Selenium + TestNG Project

Step 5: Add TestNG Suite File (testng.xml): Create a testng.xml at project root

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Demoblaze Test Suite">
  <test name="SmokeTests">
    <classes>
      <class name="tests.VerifyHomePageTitle" />
    </classes>
  </test>
</suite>
```

Step-by-Step: Creating a Maven + Selenium + TestNG Project

Step 6: Run the Project

- You can run in two ways:
 1. From Eclipse: Right-click **testng.xml** → Run As → **TestNG Suite**
 2. From Command Line (Maven):

mvn clean test

- Maven compiles your code, runs TestNG tests, and generates reports automatically in:

test-output/index.html

Maven Surefire Reports

- The **Maven Surefire Plugin** is the **default test runner** that Maven uses to execute your **TestNG** or **JUnit** test classes.
- When you run: **mvn test**
- The **Surefire Plugin** automatically runs your test cases, captures the results, and generates **test execution reports** in both **text (console)** and **HTML/XML** formats.

Importance in Real Projects

Purpose	Explanation
Automation Traceability	Provides a structured, timestamped record of every test execution.
CI/CD Integration	Jenkins reads Surefire's XML reports to display "Passed / Failed" builds.
Regression Validation	Helps QA teams quickly identify failed test cases after a new code push.
Documentation & Auditing	Creates formal evidence of executed tests (used in QA audits).

Adding Surefire Plugin in pom.xml

- Make sure this plugin is included (even though it's added by default in newer Maven versions):

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
      <configuration>
        <!-- Optional: specify your TestNG suite -->
        <suiteXmlFiles>
          <suiteXmlFile>testng.xml</suiteXmlFile>
        </suiteXmlFiles>
        <testFailureIgnore>>false</testFailureIgnore>
        <reportsDirectory>${project.build.directory}/surefire-reports</reportsDirectory>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Maven Basics

How It Works

- When you execute: **mvn test**

Here's what happens internally

1. Maven runs the Surefire Plugin.
2. The plugin searches for all classes under `src/test/java` containing methods with `@Test`.
3. TestNG or JUnit runs those test methods.
4. The plugin records results (pass/fail/skip, time, stack trace).
5. Reports are generated automatically under: **target/surefire-reports/**

Folder Structure Example

```
MyAutomationProject/  
├─ pom.xml  
├─ src/  
│   └─ test/java/  
│       └─ tests/  
│           └─ LoginTest.java  
└─ target/  
    └─ surefire-reports/  
        ├─ TEST-tests.LoginTest.xml  
        ├─ TEST-tests.LoginTest.txt  
        ├─ index.html (optional)  
        └─ summary.txt
```

Sample Execution Output (in console)

```
-----  
T E S T S  
-----  
  
Running tests.LoginTest  
Tests run: 5, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 10.123 s <<< FAIL  
testLoginWithInvalidCredentials(tests.LoginTest)  
java.lang.AssertionError: Expected login to fail
```

Sample Execution Output – After completion

```
[INFO] BUILD FAILURE  
[INFO] Surefire report directory: /target/surefire-reports
```

Maven Basics

Real IT Example (Scenario)

- Let's simulate an end-to-end build-release pipeline scenario:

Team	Step	Tool
 Developer	Writes code, commits to Git	Git
 Jenkins	Runs <code>mvn clean test package</code>	Maven
 QA Team	Verifies build success	TestNG + Reports
 Jenkins	Runs <code>mvn deploy</code>	Maven
 Artifact Repository	Uploads <code>.jar</code> to <code>https://nexus.company.com/repository/releases/</code>	Nexus
 Other Teams	Download same build (version-controlled)	Nexus UI / Maven dependency

THANK YOU